

UNIVERSITY OF THE ARMED FORCES - ESPE

Computer Science Department

Object Oriented Programming

36863

Tomas Tapia

Homework05 - UMLReading

Slides 1.01 01

Slides 1.05 07

Slides 1.05 08

Slides 1.05 09

Slides 1.05 10

LASCANO, JORGE EDISON

October 14, 2026

Summary Slides

1.05.01 - UML Foundations

Capture real-world concepts, analize, prototype & communicate design choices w/ stakeholders. It helps visualize current workflows guide implementation w/o domain in low-level details

Used thee ~~life~~ cycle

* Abstraction: focus on relevant ideas while omitting unnecessary complexity, realized via abstract classes, methods or public Interfaces
* Encapsulation: Hide the decisions behind well defined interfaces so other components cannot impacted
* Aim for loose coupling & high cohesion

1.05.02 - UML Best Practices

* Model the problem domain as stakeholders, not raw code or classes
* As u discover use cases & classes, let use follow up questions to explore domain rules
* Work w/ model like an essay, when each diagram acts as a paragraph, Try to keep diagrams clean.

1.0

L05.08 - Associations & Multiplicity constraints

Associations: Represent sets of persistent semantic links between instance of classes.

* Use clear phrases or full sentences, w/ an optional reading direction arrow instead of generic terms
* Can label to provide alternate names for an object from the connected classes per

Multiplicity constraints: spec how many objs of one class can link to a single obj of another

* Attributes & associations: early analysis, don't rush to make properties simple attributes inside a class box; then convert them to explicit associations w/ full multiplicity.

1.05.09 : ACD

① Aggregation (white)

* Represents a "part of" where the child can exist indepndly of the whole.

* Semantically reflexive, asymmetric & transitive

② Composition (Black)

* A strict, whole-part relationship

* If the parent object is destroyed, all its parts are destroyed w/ it.

③ Dependencies (- - ▷)

* One class depends on/uses another, w/o persistent data link

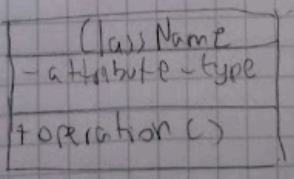
* Common names <<Use>>, <<Create>>, <<Delete>>
Keep them minimal.

1.05.10 - Class Diagrams & Use Case Diagram

Class Diagrams

Classes represents sets of domain concepts w/ attributes + operations

Draws like



* Specialization (Δ)
  Models "B is an A", where sub-class B inherits all properties & behaviors of super class A
* Binary links two classes; N-ary associations use a ◇ to link 3 or more classes w/ descriptive labels.

Use Cases Diagrams

* Actors (stick fig): Named using capitalized singular nouns
* Use cases (oval): Name using verb phrases, describe the goal
  Actor to use cases: Solid line w/o arrows with actors participating with goal
  Dependencies:
  <<include>> A cannot be completed w/o goal B
  <<extend>> Optional or exceptional use case under specific conditions (arrow points back to the base case)

Generalization: actor or use case inherits and refines the parents baseline interactions.